

Synechron Campus Placement | May 2026

HackerRank Test Strategy (1 Hour)	
• Read all problems first (2 min)	→ <i>Easiest problem = attempt first</i>
• Always save after completing each problem	→ <i>Don't wait till the end</i>
• Write clean I/O — Scanner for input	→ <i>System.out.println for output</i>
• If stuck >15 min, move to next problem	→ <i>Partial marks beat zero</i>
• Test with edge cases: empty, null, large	→ <i>Before submitting</i>

TABLE OF CONTENTS

Chapter 1: Object-Oriented Programming (OOP)

- Classes & Objects
- Constructors
- Inheritance
- Polymorphism
- Interfaces & Abstract Classes
- Encapsulation

Chapter 2: Java Collections Framework

- ArrayList
- LinkedList
- HashMap
- HashSet
- TreeMap & TreeSet
- Iterators

Chapter 3: String Manipulation

- String vs StringBuilder vs StringBuffer
- Key String Methods
- Pattern Problems
- String Formatting

Chapter 4: Exception Handling

- try-catch-finally
- Checked vs Unchecked
- Custom Exceptions
- Multi-catch

Chapter 5: Sorting & Searching Algorithms

- Bubble, Selection, Insertion Sort
- Merge Sort, Quick Sort
- Binary Search
- Comparator & Comparable

Chapter 6: Data Structures — DSA Essentials

- Arrays
- Linked List
- Stack
- Queue
- Practice Problems

CH 1

Object-Oriented Programming

Classes · Inheritance · Polymorphism · Interfaces · Abstract Classes

What is OOP and Why Does Java Use It?

Object-Oriented Programming organises code around objects — self-contained units that combine data (fields) and behaviour (methods). Java is fundamentally OOP. Every piece of Java code lives inside a class. OOP gives us four core pillars: Encapsulation, Inheritance, Polymorphism, and Abstraction. Understanding these deeply is critical because HackerRank OOP problems test all four.

1.1 Classes and Objects

A class is a blueprint — it defines what data an object holds and what operations it can perform. An object is a runtime instance of that class allocated in heap memory.

Class & Object Example

```
// Class definition
public class Student {
    // Fields (instance variables)
    private String name;
    private int age;
    private double gpa;

    // Constructor
    public Student(String name, int age, double gpa) {
        this.name = name; // 'this' refers to current object
        this.age = age;
        this.gpa = gpa;
    }

    // Getter (Encapsulation)
    public String getName() { return name; }
    public int getAge() { return age; }

    // Setter
    public void setGpa(double gpa) {
        if (gpa >= 0.0 && gpa <= 10.0) this.gpa = gpa;
    }

    // Method
    public void display() {
        System.out.println("Name: " + name + ", Age: " + age + ", GPA: " + gpa);
    }
}
```

```
}

// Creating objects
Student s1 = new Student("Nitesh", 22, 8.5);
s1.display(); // Output: Name: Nitesh, Age: 22, GPA: 8.5
```

NOTE: Use 'this' keyword to distinguish instance variables from constructor parameters when they have the same name.

1.2 Constructors

A constructor initialises an object when it is created. Its name must match the class name and it has no return type. Java provides a default no-arg constructor if you don't define any, but once you define one constructor, the default disappears.

Constructor Overloading

```
public class Box {
    double length, width, height;

    // Default constructor
    public Box() {
        length = width = height = 1.0;
    }

    // Parameterised constructor
    public Box(double l, double w, double h) {
        this.length = l; this.width = w; this.height = h;
    }

    // Constructor chaining using this()
    public Box(double side) {
        this(side, side, side); // calls parameterised constructor
    }

    public double volume() { return length * width * height; }
}
```

1.3 Inheritance

Inheritance allows a child class (subclass) to acquire properties and methods of a parent class (superclass). Use the 'extends' keyword. Java supports single inheritance for classes — a class can extend only one parent. The 'super' keyword accesses the parent's constructor or methods.

Inheritance Example

```
// Parent class
public class Animal {
    String name;

    public Animal(String name) { this.name = name; }

    public void eat() { System.out.println(name + " is eating"); }

    public void sound() { System.out.println("Some sound"); }
}
```

```
// Child class
public class Dog extends Animal {
    String breed;

    public Dog(String name, String breed) {
        super(name); // calls Animal's constructor
        this.breed = breed;
    }

    @Override
    public void sound() {
        System.out.println(name + " says: Woof!");
    }

    public void fetch() { System.out.println(name + " fetches the ball!"); }
}

// Usage
Dog d = new Dog("Bruno", "Labrador");
d.eat(); // Inherited from Animal
d.sound(); // Overridden – Bruno says: Woof!
d.fetch(); // Dog-specific
```

NOTE: Key rule: Constructor is NOT inherited. You must call `super()` explicitly if the parent has a parameterised constructor. If `super()` is not called, Java automatically inserts `super()` (no-arg) as first line.

Keyword	Purpose	Example
<code>extends</code>	Class inherits from another class	<code>class Dog extends Animal</code>
<code>super()</code>	Calls parent constructor	<code>super(name);</code>
<code>super.method()</code>	Calls parent method	<code>super.sound();</code>
<code>@Override</code>	Annotation — marks method override	<code>@Override public void sound()</code>
<code>final class</code>	Cannot be extended	<code>public final class String</code>

1.4 Polymorphism

Polymorphism means 'many forms'. In Java there are two types: Compile-time polymorphism (Method Overloading) and Runtime polymorphism (Method Overriding). Understanding the difference is a frequent interview and test question.

Method Overloading (Compile-time)

Same method name, different parameter list (type, number, or order). Resolved at compile time.

Method Overloading

```
public class Calculator {  
    // Overloaded methods – same name, different signatures  
    public int add(int a, int b) { return a + b; }  
    public double add(double a, double b) { return a + b; }  
    public int add(int a, int b, int c) { return a + b + c; }  
    public String add(String a, String b) { return a + b; }  
}  
  
Calculator c = new Calculator();  
c.add(2, 3); // calls int version -> 5  
c.add(2.5, 3.5); // calls double version -> 6.0  
c.add("Hello", "!"); // calls String version -> Hello!
```

Method Overriding (Runtime / Dynamic Dispatch)

Child class redefines a parent class method with the SAME signature. The decision of which method to call is made at runtime based on the actual object type, not the reference type.

Runtime Polymorphism

```
Animal a = new Dog("Bruno", "Lab"); // Parent ref, Child object  
a.sound(); // Calls Dog's sound() at runtime – 'Bruno says: Woof!'  
  
// This is Runtime Polymorphism / Dynamic Method Dispatch  
  
// Rules for Overriding:  
// 1. Same method name, same parameters, same return type  
// 2. Access modifier can be SAME or MORE permissive (not restrictive)  
// 3. Cannot override final or static methods  
// 4. Use @Override annotation always (good practice, catches errors)
```

TIP: Overloading = same class, different params. Overriding = parent-child, same signature. This is THE most asked distinction.

1.5 Interfaces

An interface is a 100% abstract contract — it defines WHAT a class must do, not HOW. Classes implement interfaces using the 'implements' keyword. A class can implement multiple interfaces (solving Java's single-inheritance limitation). From Java 8, interfaces can have default and static methods.

Interface Example

```
// Interface definition  
public interface Drawable {  
    void draw(); // abstract by default
```

```

default void display() { // default method (Java 8+)
System.out.println("Displaying shape");
}
}

public interface Resizable {
void resize(double factor);
}

// Class implements MULTIPLE interfaces
public class Circle implements Drawable, Resizable {
double radius;

public Circle(double r) { this.radius = r; }

@Override
public void draw() { System.out.println("Drawing circle r=" + radius); }

@Override
public void resize(double factor) { radius *= factor; }
}

// Polymorphic usage
Drawable d = new Circle(5.0);
d.draw(); // Drawing circle r=5.0
d.display(); // default method – Displaying shape

```

1.6 Abstract Classes

An abstract class is a partially implemented class that cannot be instantiated directly. It can have both abstract methods (no body) and concrete methods (with body). Use abstract classes when you want to provide some common behaviour while forcing subclasses to implement specifics.

Abstract Class

```

public abstract class Shape {
String color;

public Shape(String color) { this.color = color; }

// Abstract – subclasses MUST implement
public abstract double area();
public abstract double perimeter();

// Concrete – shared behaviour
public void printInfo() {
System.out.println("Shape: " + color + " | Area: " + area());
}
}

public class Rectangle extends Shape {
double length, width;

public Rectangle(String color, double l, double w) {

```

```

    super(color); this.length = l; this.width = w;
}
@Override public double area() { return length * width; }
@Override public double perimeter() { return 2 * (length + width); }
}

Shape s = new Rectangle("Red", 4, 5);
s.printInfo(); // Shape: Red | Area: 20.0

```

Feature	Interface	Abstract Class
Methods	Abstract + default/static (Java 8+)	Abstract + Concrete
Variables	public static final (constants)	Any type
Constructor	Not allowed	Allowed
Multiple	A class can implement many	Only one can be extended
Use when	Unrelated classes share behaviour	Related classes share state/code

CH 2

Java Collections Framework

ArrayList · LinkedList · HashMap · HashSet · TreeMap

The Collections Framework provides ready-made data structures. For HackerRank, you MUST know how to create, add, remove, search, and iterate through each collection. Always import java.util.*

TIP: Always import: import java.util.*; — this covers all collections.

2.1 ArrayList

ArrayList is a resizable array. Elements are stored in insertion order, duplicates are allowed, null values allowed. Fast random access $O(1)$, slow insert/delete in middle $O(n)$.

ArrayList — All Key Operations

```
import java.util.*;

ArrayList<String> list = new ArrayList<>();

// Add elements
list.add("Java");
list.add("Python");
list.add("C++");
list.add(1, "Kotlin"); // Insert at index 1

// Access
System.out.println(list.get(0)); // Java
System.out.println(list.size()); // 4

// Update
list.set(2, "Rust");

// Remove
list.remove("C++"); // by value
list.remove(0); // by index

// Iterate
for (String lang : list) {
    System.out.println(lang);
}

// Search
System.out.println(list.contains("Java")); // true
System.out.println(list.indexOf("Python")); // index or -1

// Sort
Collections.sort(list);
```

```
Collections.sort(list, Collections.reverseOrder());

// Convert to Array
String[] arr = list.toArray(new String[0]);
```

2.2 LinkedList

LinkedList implements both List and Deque interfaces. Elements are stored as nodes — each pointing to the next. Faster insert/delete at ends $O(1)$, slower random access $O(n)$. Used as Stack or Queue.

LinkedList Operations

```
LinkedList<Integer> ll = new LinkedList<>();

ll.add(10);
ll.add(20);
ll.add(30);

// Deque operations (Stack/Queue usage)
ll.addFirst(5); // add to front
ll.addLast(40); // add to end
ll.removeFirst(); // remove from front
ll.removeLast(); // remove from end

System.out.println(ll.peekFirst()); // see front without removing
System.out.println(ll.peekLast()); // see end without removing

// Use as Stack
ll.push(100); // same as addFirst()
ll.pop(); // same as removeFirst()

// Use as Queue
ll.offer(50); // same as addLast()
ll.poll(); // same as removeFirst()
```

2.3 HashMap

HashMap stores key-value pairs. Keys must be unique; values can repeat. No guaranteed order. Average $O(1)$ for get, put, remove. Null key allowed (one). Most used collection in HackerRank — frequency counting, grouping, memoisation.

HashMap — Complete Guide

```
HashMap<String, Integer> map = new HashMap<>();

// Put (add/update)
map.put("Alice", 90);
map.put("Bob", 85);
map.put("Charlie", 92);
map.put("Alice", 95); // overwrites Alice's value

// Get
System.out.println(map.get("Alice")); // 95
System.out.println(map.getOrDefault("Zara", 0)); // 0 if not found
```

```

// Check existence
System.out.println(map.containsKey("Bob")); // true
System.out.println(map.containsValue(85)); // true

// Remove
map.remove("Bob");

// Iterate – keySet
for (String key : map.keySet()) {
    System.out.println(key + " -> " + map.get(key));
}

// Iterate – entrySet (preferred, more efficient)
for (Map.Entry<String, Integer> entry : map.entrySet()) {
    System.out.println(entry.getKey() + ": " + entry.getValue());
}

// Common pattern: frequency count
String s = "hello world";
HashMap<Character, Integer> freq = new HashMap<>();
for (char c : s.toCharArray()) {
    freq.put(c, freq.getOrDefault(c, 0) + 1);
}

```

TIP: `getOrDefault(key, 0)` is your best friend for frequency counting. Avoid `NullPointerException` on `get()`.

2.4 HashSet

HashSet stores unique elements only. No ordering guaranteed. Backed by HashMap internally. Use when you need fast duplicate detection or set operations (union, intersection).

HashSet — Unique Elements & Set Operations

```

HashSet<Integer> set = new HashSet<>();

set.add(1); set.add(2); set.add(3); set.add(2); // duplicate ignored
System.out.println(set.size()); // 3
System.out.println(set.contains(2)); // true
set.remove(1);

// Find duplicates in array
int[] arr = {1, 2, 3, 2, 4, 1, 5};
HashSet<Integer> seen = new HashSet<>();
for (int num : arr) {
    if (!seen.add(num)) { // add() returns false if already present
        System.out.println("Duplicate: " + num);
    }
}

// Set operations
HashSet<Integer> setA = new HashSet<>(Arrays.asList(1,2,3,4));

```

```

HashSet<Integer> setB = new HashSet<>(Arrays.asList(3,4,5,6));

// Union
HashSet<Integer> union = new HashSet<>(setA);
union.addAll(setB);

// Intersection
HashSet<Integer> inter = new HashSet<>(setA);
inter.retainAll(setB);

// Difference
HashSet<Integer> diff = new HashSet<>(setA);
diff.removeAll(setB);

```

2.5 TreeMap & TreeSet (Sorted Collections)

TreeMap stores key-value pairs in SORTED key order (natural or custom Comparator). TreeSet stores unique elements in sorted order. Both use Red-Black Tree internally — $O(\log n)$ operations.

TreeMap & TreeSet

```

TreeMap<String, Integer> tm = new TreeMap<>();
tm.put("Banana", 3);
tm.put("Apple", 5);
tm.put("Cherry", 2);
// Automatically sorted by key
System.out.println(tm); // {Apple=5, Banana=3, Cherry=2}

System.out.println(tm.firstKey()); // Apple
System.out.println(tm.lastKey()); // Cherry

TreeSet<Integer> ts = new TreeSet<>();
ts.add(5); ts.add(1); ts.add(3); ts.add(1);
System.out.println(ts); // [1, 3, 5] – sorted, no duplicates
System.out.println(ts.first()); // 1
System.out.println(ts.last()); // 5

```

Collection	Order	Duplicates	Null	Complexity	Use Case
ArrayList	Insertion	Yes	Yes	$O(1)$ access	General list
LinkedList	Insertion	Yes	Yes	$O(1)$ add/remove ends	Queue/Stack
HashMap	None	Keys: No	1 null key	$O(1)$ avg	Key-value lookup
HashSet	None	No	Yes	$O(1)$ avg	Unique elements
TreeMap	Sorted	Keys: No	No	$O(\log n)$	Sorted key-value
TreeSet	Sorted	No	No	$O(\log n)$	Sorted unique

CH 3

String Manipulation

Most frequently tested topic in HackerRank Java tests

String vs StringBuilder vs StringBuffer

String in Java is immutable — every modification creates a new String object. StringBuilder is mutable and NOT thread-safe (use in single-thread — HackerRank). StringBuffer is mutable and thread-safe but slower. For almost all HackerRank problems, use StringBuilder.

String vs StringBuilder

```
// String immutability demo
String s = "Hello";
s = s + " World"; // creates NEW String object, old one GC'd

// StringBuilder — efficient for concatenation in loops
StringBuilder sb = new StringBuilder();
sb.append("Hello");
sb.append(" ");
sb.append("World");
sb.insert(5, ","); // Hello, World
sb.delete(5, 6); // Hello World
sb.reverse(); // dlrow olleH
System.out.println(sb.toString());

// Never do this in a loop — O(n^2) time:
String result = "";
for (int i = 0; i < 1000; i++) result += i; // BAD

// Do this instead — O(n) time:
StringBuilder sb2 = new StringBuilder();
for (int i = 0; i < 1000; i++) sb2.append(i); // GOOD
```

Essential String Methods

All Key String Methods

```
String str = " Hello, Java World! ";

// Length & access
str.length() // 22
str.charAt(7) // J
str.indexOf('J') // 7 (first occurrence)
str.lastIndexOf('o') // 16
```

```

// Case
str.toLowerCase() // ' hello, java world! '
str.toUpperCase() // ' HELLO, JAVA WORLD! '

// Trim & Strip
str.trim() // 'Hello, Java World!' (removes leading/trailing spaces)
str.strip() // same but handles Unicode spaces (Java 11+)

// Substring
str.trim().substring(7) // 'Java World!'
str.trim().substring(7, 11) // 'Java'

// Contains / starts / ends
str.contains("Java") // true
str.startsWith(" Hello") // true
str.endsWith("! ") // true

// Replace
str.replace("Java", "Python") // replaces first match
str.replaceAll("\\s+", " ") // regex: collapse whitespace
str.replaceFirst("[A-Z]", "X") // replace first uppercase

// Split
String csv = "a,b,c,d";
String[] parts = csv.split(","); // ["a","b","c","d"]

// Comparisons
str.equals("Hello") // content equality (use this, not ==)
str.equalsIgnoreCase("hello, java world!") // case-insensitive
str.compareTo("Hello") // 0 if equal (lexicographic)

// Convert
String.valueOf(42) // int -> String
Integer.parseInt("42") // String -> int
Double.parseDouble("3.14") // String -> double
str.toCharArray() // String -> char[]
String.join("-", "a","b","c") // "a-b-c"

```

Common String Problems & Patterns

Pattern 1: Check Palindrome

```

public static boolean isPalindrome(String s) {
    s = s.toLowerCase().replaceAll("[^a-z0-9]", "");
    int left = 0, right = s.length() - 1;
    while (left < right) {
        if (s.charAt(left) != s.charAt(right)) return false;
        left++; right--;
    }
}

```

```
return true;
}
// isPalindrome("A man a plan a canal Panama") -> true
```

Pattern 2: Check Anagram

```
public static boolean isAnagram(String s1, String s2) {
    if (s1.length() != s2.length()) return false;
    int[] count = new int[256];
    for (char c : s1.toCharArray()) count[c]++;
    for (char c : s2.toCharArray()) count[c]--;
    for (int val : count) if (val != 0) return false;
    return true;
}
// isAnagram("listen", "silent") -> true
```

Pattern 3: Reverse Words in a Sentence

```
public static String reverseWords(String sentence) {
    String[] words = sentence.trim().split("\\s+");
    StringBuilder sb = new StringBuilder();
    for (int i = words.length - 1; i >= 0; i--) {
        sb.append(words[i]);
        if (i > 0) sb.append(" ");
    }
    return sb.toString();
}
// reverseWords("Hello Java World") -> "World Java Hello"
```

Pattern 4: Count Vowels and Consonants

```
public static void countVowelsCons(String s) {
    s = s.toLowerCase();
    int vowels = 0, cons = 0;
    String vowelSet = "aeiou";
    for (char c : s.toCharArray()) {
        if (Character.isLetter(c)) {
            if (vowelSet.indexOf(c) != -1) vowels++;
            else cons++;
        }
    }
    System.out.println("Vowels: " + vowels + " Consonants: " + cons);
}
```

CH 4

Exception Handling

try-catch-finally · Checked vs Unchecked · Custom Exceptions

An exception is a runtime event that disrupts normal program flow. Java uses a try-catch mechanism to handle exceptions gracefully — preventing program crashes and providing useful error messages.

4.1 Exception Hierarchy

All exceptions extend Throwable. Two main branches: Error (JVM-level, don't catch) and Exception (application-level, handle).

Exception Hierarchy

```
Throwable
  ■■■ Error (don't catch these – JVM issues)
    ■ ■■■ OutOfMemoryError
    ■ ■■■ StackOverflowError
  ■■■ Exception
    ■■■ RuntimeException (Unchecked – not required to catch)
      ■ ■■■ NullPointerException
      ■ ■■■ ArrayIndexOutOfBoundsException
      ■ ■■■ ClassCastException
      ■ ■■■ NumberFormatException
      ■ ■■■ ArithmeticException (division by zero)
    ■■■ Checked Exceptions (MUST catch or declare throws)
      ■■■ IOException
      ■■■ FileNotFoundException
      ■■■ SQLException
```

4.2 try-catch-finally

try-catch-finally

```
public static void divide(int a, int b) {
    try {
        int result = a / b; // may throw ArithmeticException
        System.out.println("Result: " + result);
    } catch (ArithmeticException e) {
        System.out.println("Cannot divide by zero: " + e.getMessage());
    } catch (Exception e) { // catches any other exception
        System.out.println("Unexpected error: " + e.getMessage());
    } finally {
```



```
// ALWAYS executes – even if exception thrown or return called
System.out.println("Execution complete.");
}
}

divide(10, 2); // Result: 5 | Execution complete.
divide(10, 0); // Cannot divide by zero: / by zero | Execution complete.
```

NOTE: finally block always runs — use it for closing resources (file, DB connection, etc.)

4.3 Multi-catch and try-with-resources

Multi-catch

```
// Multi-catch (Java 7+) – handle multiple exceptions same way
try {
    int[] arr = new int[5];
    arr[10] = Integer.parseInt("abc"); // two potential exceptions
} catch (ArrayIndexOutOfBoundsException | NumberFormatException e) {
    System.out.println("Either index or format error: " + e.getMessage());
}

// try-with-resources – auto-closes resources
try (Scanner sc = new Scanner(System.in)) {
    int n = sc.nextInt();
    System.out.println(n * 2);
} // sc.close() called automatically
```

4.4 Custom Exceptions

You can create your own exception classes by extending `Exception` (checked) or `RuntimeException` (unchecked). Custom exceptions make your code self-documenting and allow specific error handling.

Custom Exception

```
// Custom checked exception
public class InsufficientFundsException extends Exception {
    private double amount;

    public InsufficientFundsException(double amount) {
        super("Insufficient funds. Shortage: " + amount);
        this.amount = amount;
    }

    public double getAmount() { return amount; }
}

public class BankAccount {
    private double balance;

    public BankAccount(double balance) { this.balance = balance; }

    public void withdraw(double amount) throws InsufficientFundsException {
        if (amount > balance) {
```

```

    throw new InsufficientFundsException(amount - balance);
}
balance -= amount;
}
}

// Usage
BankAccount acc = new BankAccount(100.0);
try {
    acc.withdraw(150.0);
} catch (InsufficientFundsException e) {
    System.out.println(e.getMessage()); // Insufficient funds. Shortage: 50.0
}

```

Exception	Type	When it occurs
NullPointerException	Unchecked	Calling method on null reference
ArrayIndexOutOfBoundsException	Unchecked	Accessing arr[-1] or arr[arr.length]
NumberFormatException	Unchecked	Integer.parseInt("abc")
ArithmeticException	Unchecked	Division by zero
ClassCastException	Unchecked	Illegal cast between types
IOException	Checked	File read/write failure
StackOverflowError	Error	Infinite/deep recursion

CH 5

Sorting & Searching Algorithms

Bubble · Selection · Insertion · Merge · Quick Sort · Binary Search

Sorting is one of the most tested areas in HackerRank. You must know: how each algorithm works conceptually, its time and space complexity, and when to use which. Also know Java's built-in `Arrays.sort()` and `Collections.sort()` with custom Comparators.

Algorithm	Best	Average	Worst	Space	Stable
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
<code>Arrays.sort()</code>	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes

5.1 Bubble Sort

Repeatedly compares adjacent elements and swaps if in wrong order. After each pass, the largest unsorted element bubbles to the end. Simple but $O(n^2)$ — only use for small arrays.

Bubble Sort

```
public static void bubbleSort(int[] arr) {
    int n = arr.length;
    for (int i = 0; i < n - 1; i++) {
        boolean swapped = false;
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                // Swap
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swapped = true;
            }
        }
        if (!swapped) break; // Optimisation: already sorted
    }
}
```

```
// Input: [64, 34, 25, 12, 22]
// Output: [12, 22, 25, 34, 64]
```

5.2 Selection Sort

Finds the minimum element in the unsorted portion and places it at the beginning. Always $O(n^2)$ — doesn't benefit from partially sorted input.

Selection Sort

```
public static void selectionSort(int[] arr) {
    int n = arr.length;
    for (int i = 0; i < n - 1; i++) {
        int minIdx = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIdx]) minIdx = j;
        }
        // Swap min with first unsorted
        int temp = arr[minIdx];
        arr[minIdx] = arr[i];
        arr[i] = temp;
    }
}
```

5.3 Insertion Sort

Builds sorted array one element at a time. Very efficient for nearly-sorted data. Like sorting playing cards in hand.

Insertion Sort

```
public static void insertionSort(int[] arr) {
    int n = arr.length;
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;
        // Shift elements greater than key one position right
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}
```

5.4 Merge Sort

Divide and conquer: divide array in half recursively, sort each half, then merge. Guaranteed $O(n \log n)$ always. Stable sort. Used in Java's `Arrays.sort()` for objects.

Merge Sort

```
public static void mergeSort(int[] arr, int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2; // avoids overflow
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

private static void merge(int[] arr, int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;
    int[] L = new int[n1], R = new int[n2];

    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) arr[k++] = L[i++];
        else arr[k++] = R[j++];
    }
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}

// Call: mergeSort(arr, 0, arr.length - 1);
```

5.5 Quick Sort

Picks a pivot, partitions array so all elements $<$ pivot are left of it and all $>$ pivot are right. Recursively sorts each partition. Average $O(n \log n)$, worst $O(n^2)$ on sorted arrays.

Quick Sort

```
public static void quickSort(int[] arr, int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

private static int partition(int[] arr, int low, int high) {
    int pivot = arr[high]; // choose last element as pivot
```

```

int i = low - 1; // index of smaller element
for (int j = low; j < high; j++) {
    if (arr[j] <= pivot) {
        i++;
        int temp = arr[i]; arr[i] = arr[j]; arr[j] = temp;
    }
}
int temp = arr[i+1]; arr[i+1] = arr[high]; arr[high] = temp;
return i + 1;
}

// Call: quickSort(arr, 0, arr.length - 1);

```

5.6 Java Built-in Sorting — Arrays.sort() and Comparator

For HackerRank, prefer Java's built-in sort for efficiency. Know how to write Comparators for custom sorting.

Built-in Sort & Comparators

```

// Sort primitive array
int[] arr = {5, 2, 8, 1, 9};
Arrays.sort(arr); // [1, 2, 5, 8, 9]
Arrays.sort(arr, 1, 4); // sort subarray [1,4) only

// Sort in reverse – must use Integer[] not int[]
Integer[] boxed = {5, 2, 8, 1, 9};
Arrays.sort(boxed, Collections.reverseOrder());

// Sort ArrayList
ArrayList<Integer> list = new ArrayList<>(Arrays.asList(5,2,8,1));
Collections.sort(list); // ascending
Collections.sort(list, (a, b) -> b - a); // descending (lambda)

// Sort objects with Comparator
ArrayList<String> names = new ArrayList<>(Arrays.asList("Zara","Alice","Bob"));
Collections.sort(names, (a, b) -> a.length() - b.length()); // sort by length

// Sort by multiple fields
// Assume: class Student { String name; int grade; }
students.sort((a, b) -> {
    if (a.grade != b.grade) return b.grade - a.grade; // desc grade
    return a.name.compareTo(b.name); // then asc name
});

```

5.7 Binary Search

Binary Search works ONLY on sorted arrays. Repeatedly divides the search space in half. $O(\log n)$ time. Know both iterative and recursive versions.

Binary Search

```

// Iterative Binary Search

```

```
public static int binarySearch(int[] arr, int target) {
    int left = 0, right = arr.length - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2; // prevents overflow
        if (arr[mid] == target) return mid; // found
        else if (arr[mid] < target) left = mid + 1; // search right
        else right = mid - 1; // search left
    }
    return -1; // not found
}

// Java built-in binary search
int[] sorted = {1, 3, 5, 7, 9, 11};
int idx = Arrays.binarySearch(sorted, 7); // returns 3
int notFound = Arrays.binarySearch(sorted, 6); // returns negative

// Binary Search on ArrayList
ArrayList<Integer> list = new ArrayList<>(Arrays.asList(1,3,5,7,9));
int pos = Collections.binarySearch(list, 5); // returns 2
```

WARNING: Binary search: always use $\text{mid} = \text{left} + (\text{right} - \text{left}) / 2$, NOT $(\text{left} + \text{right}) / 2$, to prevent integer overflow.

CH 6

Data Structures — DSA Essentials

Arrays · Linked List · Stack · Queue · Practice Problems

6.1 Arrays — Fundamentals & Patterns

Arrays are the most fundamental data structure. Fixed size, $O(1)$ access by index. Master these essential array patterns — they appear constantly in HackerRank.

Array Patterns

```
// Declaration & Initialisation
int[] arr = new int[5]; // [0,0,0,0,0]
int[] arr2 = {10, 20, 30, 40, 50}; // direct initialisation
int[][] matrix = new int[3][4]; // 2D array 3 rows x 4 cols

// Common operations
arr.length // size (not a method, it's a field)
Arrays.fill(arr, 0); // fill all with 0
Arrays.copyOf(arr, arr.length); // copy array
System.arraycopy(src, 0, dest, 0, n); // fast copy

// Two-pointer technique – find pair with target sum
public static int[] twoSum(int[] arr, int target) {
    int left = 0, right = arr.length - 1;
    Arrays.sort(arr); // must be sorted
    while (left < right) {
        int sum = arr[left] + arr[right];
        if (sum == target) return new int[]{left, right};
        else if (sum < target) left++;
        else right--;
    }
    return new int[]{-1, -1};
}

// Kadane's Algorithm – Maximum Subarray Sum
public static int maxSubarraySum(int[] arr) {
    int maxSoFar = arr[0], maxEndingHere = arr[0];
    for (int i = 1; i < arr.length; i++) {
        maxEndingHere = Math.max(arr[i], maxEndingHere + arr[i]);
        maxSoFar = Math.max(maxSoFar, maxEndingHere);
    }
    return maxSoFar;
}
```



```

}

// Rotate array right by k positions
public static void rotate(int[] arr, int k) {
    int n = arr.length;
    k %= n;
    reverse(arr, 0, n - 1);
    reverse(arr, 0, k - 1);
    reverse(arr, k, n - 1);
}

private static void reverse(int[] arr, int l, int r) {
    while (l < r) { int t = arr[l]; arr[l++] = arr[r]; arr[r--] = t; }
}

```

6.2 Linked List — Custom Implementation

A linked list is a chain of nodes where each node stores a value and a pointer to the next node. Unlike arrays, not stored contiguously in memory. Dynamic size — insert/delete $O(1)$ if you have the node reference.

Linked List Implementation

```

// Node class
class Node {
    int data;
    Node next;
    Node(int data) { this.data = data; this.next = null; }
}

// Singly Linked List
class LinkedList {
    Node head;

    // Insert at end
    public void append(int data) {
        Node newNode = new Node(data);
        if (head == null) { head = newNode; return; }
        Node curr = head;
        while (curr.next != null) curr = curr.next;
        curr.next = newNode;
    }

    // Insert at beginning
    public void prepend(int data) {
        Node newNode = new Node(data);
        newNode.next = head;
        head = newNode;
    }

    // Delete by value

```

```
public void delete(int data) {
    if (head == null) return;
    if (head.data == data) { head = head.next; return; }
    Node curr = head;
    while (curr.next != null && curr.next.data != data)
        curr = curr.next;
    if (curr.next != null) curr.next = curr.next.next;
}

// Reverse linked list (iterative)
public void reverse() {
    Node prev = null, curr = head, next = null;
    while (curr != null) {
        next = curr.next;
        curr.next = prev;
        prev = curr;
        curr = next;
    }
    head = prev;
}

// Detect cycle (Floyd's algorithm)
public boolean hasCycle() {
    Node slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) return true;
    }
    return false;
}

// Find middle (slow-fast pointer)
public Node findMiddle() {
    Node slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }
    return slow;
}

// Print
public void print() {
    Node curr = head;
```

```
while (curr != null) {  
    System.out.print(curr.data + " -> ");  
    curr = curr.next;  
}  
System.out.println("null");  
}  
}
```

6.3 Stack — LIFO Data Structure

Stack follows LIFO (Last In, First Out). Push adds to top, pop removes from top. Used for: balanced parentheses, expression evaluation, undo functionality, recursion simulation.

Stack — Operations & Problems

```
import java.util.Stack;

Stack<Integer> stack = new Stack<>();

// Basic operations
stack.push(10); // add to top
stack.push(20);
stack.push(30);
stack.peek(); // 30 — see top without removing
stack.pop(); // 30 — remove from top
stack.isEmpty(); // false
stack.size(); // 2

// OR use Deque (preferred over Stack class in modern Java)
Deque<Integer> dstack = new ArrayDeque<>();
dstack.push(10); dstack.push(20);
dstack.peek(); // 20
dstack.pop(); // 20

// Classic problem: Balanced Parentheses
public static boolean isBalanced(String s) {
    Deque<Character> stack2 = new ArrayDeque<>();
    for (char c : s.toCharArray()) {
        if (c == '(' || c == '[' || c == '{') {
            stack2.push(c);
        } else {
            if (stack2.isEmpty()) return false;
            char top = stack2.pop();
            if (c == ')' && top != '(') return false;
            if (c == ']' && top != '[') return false;
            if (c == '}' && top != '{') return false;
        }
    }
    return stack2.isEmpty();
}

// isBalanced("[{()}]") -> true
// isBalanced("[{()}]") -> false

// Problem: Next Greater Element
public static int[] nextGreater(int[] arr) {
    int n = arr.length;
    int[] result = new int[n];
```

```

Arrays.fill(result, -1);
Deque<Integer> st = new ArrayDeque<>(); // stores indices
for (int i = 0; i < n; i++) {
    while (!st.isEmpty() && arr[i] > arr[st.peek()]) {
        result[st.pop()] = arr[i];
    }
    st.push(i);
}
return result;
}

```

6.4 Queue — FIFO Data Structure

Queue follows FIFO (First In, First Out). Enqueue adds to rear, dequeue removes from front. Used for: BFS traversal, task scheduling, printer queues, sliding window problems.

Queue & PriorityQueue

```

import java.util.*;

// Queue using LinkedList
Queue<Integer> queue = new LinkedList<>();

queue.offer(10); // add to rear (preferred over add())
queue.offer(20);
queue.offer(30);

queue.peek(); // 10 – see front without removing
queue.poll(); // 10 – remove from front
queue.isEmpty(); // false
queue.size(); // 2

// PriorityQueue – elements removed in priority order
PriorityQueue<Integer> pq = new PriorityQueue<>(); // min-heap
pq.offer(30); pq.offer(10); pq.offer(20);
pq.poll(); // 10 – always returns minimum

// Max-heap
PriorityQueue<Integer> maxPq = new PriorityQueue<>(Collections.reverseOrder());
maxPq.offer(30); maxPq.offer(10); maxPq.offer(20);
maxPq.poll(); // 30 – returns maximum

// Sliding Window Maximum using Deque
public static int[] maxSlidingWindow(int[] arr, int k) {
    int n = arr.length;
    int[] result = new int[n - k + 1];
    Deque<Integer> dq = new ArrayDeque<>(); // stores indices
    for (int i = 0; i < n; i++) {
        // Remove elements outside window
        while (!dq.isEmpty() && dq.peekFirst() <= i - k) dq.pollFirst();
    }
}

```

```
// Remove smaller elements from rear
while (!dq.isEmpty() && arr[dq.peekLast()] <= arr[i]) dq.pollLast();
dq.offerLast(i);
if (i >= k - 1) result[i - k + 1] = arr[dq.peekFirst()];
}
return result;
}
```

CH 7

Practice Questions & Solutions

Top HackerRank-style problems with complete Java solutions

These are the most commonly asked problem types on HackerRank Java tests. Study each solution pattern — you won't see the exact same problem, but the underlying pattern will repeat.

Q1. FizzBuzz (Classic — Warm-up)

Print 1 to 100. For multiples of 3 print 'Fizz', multiples of 5 print 'Buzz', multiples of both print 'FizzBuzz'.

```
for (int i = 1; i <= 100; i++) {
    if (i % 15 == 0) System.out.println("FizzBuzz");
    else if (i % 3 == 0) System.out.println("Fizz");
    else if (i % 5 == 0) System.out.println("Buzz");
    else System.out.println(i);
}

// KEY: Check % 15 FIRST (both), then % 3, then % 5
```

Q2. Fibonacci Series — Iterative & Recursive

```
// Iterative — O(n) time, O(1) space — PREFERRED
public static long fibonacci(int n) {
    if (n <= 1) return n;
    long a = 0, b = 1;
    for (int i = 2; i <= n; i++) {
        long c = a + b;
        a = b; b = c;
    }
    return b;
}

// Recursive — O(2^n) — SLOW, but know the code
public static long fibRecursive(int n) {
    if (n <= 1) return n;
    return fibRecursive(n - 1) + fibRecursive(n - 2);
}

// Print first n fibonacci numbers
public static void printFib(int n) {
    long a = 0, b = 1;
    for (int i = 0; i < n; i++) {
        System.out.print(a + " ");
    }
}
```

```
long c = a + b; a = b; b = c;
}
}
```

Q3. Check Prime Number

```
public static boolean isPrime(int n) {
    if (n < 2) return false;
    if (n == 2) return true;
    if (n % 2 == 0) return false;
    // Only check up to sqrt(n) – key optimisation
    for (int i = 3; i * i <= n; i += 2) {
        if (n % i == 0) return false;
    }
    return true;
}
// isPrime(17) -> true | isPrime(18) -> false
```

Q4. Factorial — Iterative & Recursive

```
// Iterative
public static long factorial(int n) {
    long result = 1;
    for (int i = 2; i <= n; i++) result *= i;
    return result;
}

// Recursive
public static long factRecursive(int n) {
    if (n == 0 || n == 1) return 1;
    return n * factRecursive(n - 1);
}
```

Q5. Find Duplicate Elements in Array

```
public static void findDuplicates(int[] arr) {
    HashSet<Integer> seen = new HashSet<>();
    HashSet<Integer> duplicates = new HashSet<>();
    for (int num : arr) {
        if (!seen.add(num)) {
            duplicates.add(num);
        }
    }
    System.out.println("Duplicates: " + duplicates);
}
// Input: [1, 2, 3, 2, 4, 1] -> Duplicates: [1, 2]
```


Q6. Reverse a String Without Using reverse()

```
public static String reverseString(String s) {  
    char[] chars = s.toCharArray();  
    int left = 0, right = chars.length - 1;  
    while (left < right) {  
        char temp = chars[left];  
        chars[left++] = chars[right];  
        chars[right--] = temp;  
    }  
    return new String(chars);  
}  
  
// reverseString("HackerRank") -> "knaRrekcaH"
```

Q7. Count Character Frequency (Word Count)

```
public static void charFrequency(String s) {
    HashMap<Character, Integer> freq = new HashMap<>();
    for (char c : s.toCharArray()) {
        freq.put(c, freq.getOrDefault(c, 0) + 1);
    }
    // Print sorted by character
    TreeMap<Character, Integer> sorted = new TreeMap<>(freq);
    for (Map.Entry<Character, Integer> e : sorted.entrySet()) {
        System.out.println(" + + e.getKey() + " + : " + e.getValue());
    }
}
```

Q8. Find Second Largest Element in Array

```
public static int secondLargest(int[] arr) {
    int max = Integer.MIN_VALUE;
    int second = Integer.MIN_VALUE;
    for (int num : arr) {
        if (num > max) {
            second = max;
            max = num;
        } else if (num > second && num != max) {
            second = num;
        }
    }
    return second; // Integer.MIN_VALUE if no second exists
}
// Input: [12, 35, 1, 10, 34, 1] -> 34
```

Q9. Implement Stack Using Two Queues

```
class StackUsingQueues {
    Queue<Integer> q1 = new LinkedList<>();
    Queue<Integer> q2 = new LinkedList<>();

    public void push(int x) {
        q2.offer(x);
        while (!q1.isEmpty()) q2.offer(q1.poll());
        Queue<Integer> temp = q1; q1 = q2; q2 = temp;
    }

    public int pop() { return q1.poll(); }
    public int top() { return q1.peek(); }
    public boolean empty() { return q1.isEmpty(); }
}
```

Q10. Matrix Transpose

```
public static int[][] transpose(int[][] matrix) {
    int rows = matrix.length;
    int cols = matrix[0].length;
    int[][] result = new int[cols][rows];
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            result[j][i] = matrix[i][j];
        }
    }
    return result;
}

// [[1,2,3],[4,5,6]] -> [[1,4],[2,5],[3,6]]
```

Q11. OOP — Bank Account with Exception Handling

Combined OOP + Exception Handling — high probability in Synchro test.

```
class InsufficientFundsException extends Exception {
    public InsufficientFundsException(String msg) { super(msg); }
}

class BankAccount {
    private String owner;
    private double balance;

    public BankAccount(String owner, double initialBalance) {
        this.owner = owner;
        this.balance = initialBalance;
    }

    public void deposit(double amount) {
        if (amount <= 0) throw new IllegalArgumentException("Amount must be positive");
        balance += amount;
        System.out.printf("Deposited %.2f. New balance: %.2f\n", amount, balance);
    }

    public void withdraw(double amount) throws InsufficientFundsException {
        if (amount <= 0) throw new IllegalArgumentException("Amount must be positive");
        if (amount > balance) throw new InsufficientFundsException("Need " + (amount - balance) + " more");
        balance -= amount;
        System.out.printf("Withdrew %.2f. Balance: %.2f\n", amount, balance);
    }

    public double getBalance() { return balance; }
    public String toString() { return owner + ": Rs " + balance; }
}
```

```
// Main usage
BankAccount acc = new BankAccount("Nitesh", 5000.0);
try {
    acc.deposit(2000);
    acc.withdraw(3000);
    acc.withdraw(10000); // throws InsufficientFundsException
} catch (InsufficientFundsException e) {
    System.out.println("Error: " + e.getMessage());
} finally {
    System.out.println(acc);
}
```

Q12. Generic Pair Class (Generics)

```
// Generic class – works with any types
class Pair<K, V> {
    private K key;
    private V value;

    public Pair(K key, V value) { this.key = key; this.value = value; }
    public K getKey() { return key; }
    public V getValue() { return value; }

    @Override
    public String toString() { return "(" + key + ", " + value + ")"; }
}

Pair<String, Integer> p = new Pair<>("Alice", 95);
System.out.println(p); // (Alice, 95)
System.out.println(p.getKey()); // Alice
```

Q13. Collections — Group & Sort Students by Grade

```
import java.util.*;

class Student {
    String name; int grade;
    Student(String n, int g) { name = n; grade = g; }
    public String toString() { return name + "(" + grade + ")"; }
}

public class GroupStudents {
    public static void main(String[] args) {
        ArrayList<Student> students = new ArrayList<>(Arrays.asList(
            new Student("Alice", 90), new Student("Bob", 85),
            new Student("Charlie", 90), new Student("Dave", 85),
            new Student("Eve", 95)
        ));

        // Sort by grade descending, then name ascending
        students.sort((a, b) -> {
            if (b.grade != a.grade) return b.grade - a.grade;
            return a.name.compareTo(b.name);
        });

        System.out.println("Sorted: " + students);

        // Group by grade using HashMap
        HashMap<Integer, List<String>> grouped = new HashMap<>();
        for (Student s : students) {
            grouped.computeIfAbsent(s.grade, k -> new ArrayList<>()).add(s.name);
        }

        System.out.println("Grouped: " + grouped);
    }
}
```

```
}  
  
}
```

6.5 HackerRank Java I/O — Reading Input Correctly

Most HackerRank failures happen because of I/O mistakes. Always use Scanner. Know how to read different input types.

Standard HackerRank Input Template

```
import java.util.Scanner;  
  
public class Solution {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
  
        // Read single int  
        int n = sc.nextInt();  
  
        // Read array of n integers on one line  
        int[] arr = new int[n];  
        for (int i = 0; i < n; i++) arr[i] = sc.nextInt();  
  
        // Read a whole line  
        sc.nextLine(); // consume leftover newline after nextInt()  
        String line = sc.nextLine();  
  
        // Read multiple lines until EOF  
        while (sc.hasNextLine()) {  
            String l = sc.nextLine();  
            // process l  
        }  
  
        // Read integers until end of line  
        String[] parts = sc.nextLine().trim().split("\\s+");  
        int[] nums = new int[parts.length];  
        for (int i = 0; i < parts.length; i++)  
            nums[i] = Integer.parseInt(parts[i]);  
  
        sc.close();  
    }  
}
```

WARNING: After reading an int with `nextInt()`, always call `sc.nextLine()` before reading a string with `nextLine()` — otherwise `nextLine()` reads the empty newline character.

REF

Quick Reference Cheat Sheet

Everything you need at a glance before the test

Java Keywords Quick Reference

Keyword/Concept	Meaning	Example
static	Belongs to class, not instance	<code>static int count = 0;</code>
final	Cannot change (variable/method/class)	<code>final int MAX = 100;</code>
this	Current object reference	<code>this.name = name;</code>
super	Parent class reference	<code>super(name);</code>
abstract	Incomplete — must be overridden	<code>abstract void draw();</code>
interface	Contract class must implement	<code>implements Runnable</code>
extends	Class inherits from class	<code>class Dog extends Animal</code>
implements	Class fulfils interface contract	<code>class Circle implements Shape</code>
instanceof	Type check	<code>if (obj instanceof Dog)</code>
throw	Manually throw an exception	<code>throw new Exception("msg");</code>
throws	Declare method may throw exception	<code>void read() throws IOException</code>
try/catch/finally	Exception handling block	<code>try { } catch(E e){ } finally{ }</code>
synchronized	Thread-safe block	<code>synchronized(lock) { }</code>
volatile	Prevent caching in multi-thread	<code>volatile boolean flag;</code>

Access Modifiers

Modifier	Same Class	Same Package	Subclass	Everywhere
private	YES	NO	NO	NO
default (none)	YES	YES	NO	NO
protected	YES	YES	YES	NO
public	YES	YES	YES	YES

Complexity Cheat Sheet

Operation	Array	ArrayList	LinkedList	HashMap	TreeMap
Access by index	$O(1)$	$O(1)$	$O(n)$	N/A	N/A
Search	$O(n)$	$O(n)$	$O(n)$	$O(1)$ avg	$O(\log n)$
Insert at end	N/A	$O(1)$ amort	$O(1)$	$O(1)$ avg	$O(\log n)$
Insert at start	N/A	$O(n)$	$O(1)$	$O(1)$ avg	$O(\log n)$
Delete	N/A	$O(n)$	$O(1)^*$	$O(1)$ avg	$O(\log n)$
Contains/Get	$O(n)$	$O(n)$	$O(n)$	$O(1)$ avg	$O(\log n)$

4. Compile and test mentally before submitting.

5. If stu

CH 8

Advanced String Patterns

Regex · StringBuilder tricks · Pattern matching · Formatting

8.1 String Formatting

String.format() and printf() let you control how values are displayed. Very useful for HackerRank problems that require formatted output.

String Formatting

```
// String.format()
String s1 = String.format("Name: %-10s Age: %3d GPA: %.2f", "Nitesh", 22, 8.75);
// Name: Nitesh Age: 22 GPA: 8.75

// printf – prints directly
System.out.printf("%-15s %5d %8.2f\n", "Alice", 90, 9.1);
System.out.printf("%-15s %5d %8.2f\n", "Bob", 85, 8.5);
// Alice 90 9.10
// Bob 85 8.50

// Format specifiers:
// %d -> integer %f -> float/double
// %s -> string %c -> char
// %n -> newline %b -> boolean
// %-10s -> left-align in 10 chars
// %05d -> zero-pad to 5 digits (e.g., 00042)
// %.2f -> 2 decimal places

// Useful patterns
System.out.printf("%05d\n", 42); // 00042
System.out.printf("%.1f\n", 3.14); // +3.1
String hex = String.format("%X", 255); // FF (hex uppercase)
String bin = Integer.toBinaryString(10); // 1010
```

8.2 Regular Expressions in Java

Java's java.util.regex package provides Pattern and Matcher classes. Many HackerRank string problems can be solved elegantly with regex.

Regular Expressions

```
import java.util.regex.*;

// Check if string matches pattern
String email = "nitesh@example.com";
```

```

boolean valid = email.matches("[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\\.[a-zA-Z]{2,}");
System.out.println(valid); // true

// Find all numbers in a string
String text = "I have 3 cats and 12 dogs and 1 parrot";
Pattern p = Pattern.compile("\\d+");
Matcher m = p.matcher(text);
while (m.find()) {
    System.out.print(m.group() + " "); // 3 12 1
}

// Replace using regex
String s = "Hello World Java";
String clean = s.replaceAll("\\s+", " "); // "Hello World Java"

// Split on multiple delimiters
String data = "one,two;three|four";
String[] parts = data.split("[,;|]"); // ["one","two","three","four"]

// Check only digits
System.out.println("12345".matches("\\d+")); // true
System.out.println("123a5".matches("\\d+")); // false

// Common regex patterns:
// \\d+ one or more digits
// \\w+ one or more word chars (a-z, A-Z, 0-9, _)
// \\s+ one or more whitespace
// [A-Z] single uppercase letter
// ^...$ match entire string
// (a|b) a or b

```

8.3 More String Practice Problems

Pattern: Longest Common Prefix

```

public static String longestCommonPrefix(String[] strs) {
    if (strs == null || strs.length == 0) return "";
    String prefix = strs[0];
    for (int i = 1; i < strs.length; i++) {
        while (!strs[i].startsWith(prefix)) {
            prefix = prefix.substring(0, prefix.length() - 1);
            if (prefix.isEmpty()) return "";
        }
    }
    return prefix;
}

// ["flower","flow","flight"] -> "fl"

```

Pattern: Count Words in a Sentence

```

public static int countWords(String sentence) {
    if (sentence == null || sentence.trim().isEmpty()) return 0;
    return sentence.trim().split("\\s+").length;
}

// countWords(" Hello Java World ") -> 3

// Count occurrences of a word
public static int countOccurrences(String text, String word) {
    int count = 0;
    String[] words = text.toLowerCase().split("\\s+");
    for (String w : words) if (w.equals(word.toLowerCase())) count++;
    return count;
}

```

Pattern: Roman Numerals to Integer

```

public static int romanToInt(String s) {
    HashMap<Character, Integer> map = new HashMap<>();
    map.put('I',1); map.put('V',5); map.put('X',10); map.put('L',50);
    map.put('C',100); map.put('D',500); map.put('M',1000);
    int result = 0;
    for (int i = 0; i < s.length(); i++) {
        int curr = map.get(s.charAt(i));
        int next = (i+1 < s.length()) ? map.get(s.charAt(i+1)) : 0;
        if (curr < next) result -= curr;
        else result += curr;
    }
    return result;
}

// romanToInt("XIV") -> 14 | romanToInt("MCMXCIV") -> 1994

```

Pattern: First Non-Repeating Character

```

public static char firstNonRepeating(String s) {
    LinkedHashMap<Character, Integer> map = new LinkedHashMap<>();
    for (char c : s.toCharArray())
        map.put(c, map.getOrDefault(c, 0) + 1);
    for (Map.Entry<Character, Integer> e : map.entrySet())
        if (e.getValue() == 1) return e.getKey();
    return '_'; // none found
}

// firstNonRepeating("aabbcd") -> 'c'
// firstNonRepeating("aabb") -> '_'

```

CH 9

OOP Design Patterns

Singleton · Factory · Builder — common in Java tests

Design patterns are reusable solutions to common problems. Synchrotron and other product companies often ask you to implement one of these in HackerRank or technical rounds.

9.1 Singleton Pattern

Ensures only ONE instance of a class exists throughout the application. Used for: database connections, logging, configuration managers.

Singleton Pattern

```
public class Singleton {
    // Step 1: private static instance
    private static Singleton instance;

    // Step 2: private constructor – no one else can create it
    private Singleton() {
        System.out.println("Singleton created");
    }

    // Step 3: public static method to get/create instance
    public static Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }

    public void showMessage() {
        System.out.println("Hello from Singleton!");
    }
}

// Usage
Singleton s1 = Singleton.getInstance();
Singleton s2 = Singleton.getInstance();
System.out.println(s1 == s2); // true – same object
s1.showMessage();
```

9.2 Factory Pattern

Creates objects without exposing the creation logic. The client asks a factory for an object without knowing which concrete class is being used.

Factory Pattern

```
// Abstract product
interface Shape {
    void draw();
}

// Concrete products
class Circle implements Shape {
    public void draw() { System.out.println("Drawing Circle"); }
}
class Square implements Shape {
    public void draw() { System.out.println("Drawing Square"); }
}
class Triangle implements Shape {
    public void draw() { System.out.println("Drawing Triangle"); }
}

// Factory
class ShapeFactory {
    public static Shape getShape(String type) {
        switch (type.toLowerCase()) {
            case "circle": return new Circle();
            case "square": return new Square();
            case "triangle": return new Triangle();
            default: throw new IllegalArgumentException("Unknown shape: " + type);
        }
    }
}

// Client code – doesn't know about Circle/Square/Triangle classes
Shape s = ShapeFactory.getShape("circle");
s.draw(); // Drawing Circle
```

9.3 Builder Pattern

Constructs complex objects step by step. Useful when an object has many optional fields. Avoids telescoping constructors (constructors with many parameters).

Builder Pattern

```
public class Student {
    private String name; // required
    private int age; // required
    private String email; // optional
    private String branch; // optional
    private double gpa; // optional

    private Student(Builder builder) {
        this.name = builder.name;
```

```
this.age = builder.age;
this.email = builder.email;
this.branch = builder.branch;
this.gpa = builder.gpa;
}

public String toString() {
return "Student{name=" + name + ", age=" + age +
", email=" + email + ", branch=" + branch + ", gpa=" + gpa + "}";
}

// Static inner Builder class
public static class Builder {
private String name; private int age;
private String email = ""; private String branch = ""; private double gpa = 0.0;

public Builder(String name, int age) { // required fields in constructor
this.name = name; this.age = age;
}

public Builder email(String email) { this.email = email; return this; }
public Builder branch(String branch) { this.branch = branch; return this; }
public Builder gpa(double gpa) { this.gpa = gpa; return this; }
public Student build() { return new Student(this); }
}

// Usage – clean, readable, no long constructor
Student s = new Student.Builder("Nitesh", 22)
.email("nitesh@example.com")
.branch("CS")
.gpa(8.75)
.build();
System.out.println(s);
```

CH 10

Generics, Lambdas & Streams

Modern Java features tested in intermediate HackerRank problems

10.1 Generics

Generics let you write type-safe code that works with any data type. You see them everywhere: ArrayList, HashMap. Know how to write generic methods and generic classes.

Generics

```
// Generic method – works with any type T
public static <T extends Comparable<T>> T findMax(T[] arr) {
    T max = arr[0];
    for (T item : arr)
        if (item.compareTo(max) > 0) max = item;
    return max;
}

Integer[] nums = {3, 1, 4, 1, 5, 9, 2};
String[] words = {"banana", "apple", "cherry"};
System.out.println(findMax(nums)); // 9
System.out.println(findMax(words)); // cherry

// Generic stack
class GenericStack<T> {
    private ArrayList<T> items = new ArrayList<>();
    public void push(T item) { items.add(item); }
    public T pop() {
        if (isEmpty()) throw new RuntimeException("Stack is empty");
        return items.remove(items.size() - 1);
    }
    public T peek() { return items.get(items.size()-1); }
    public boolean isEmpty() { return items.isEmpty(); }
    public int size() { return items.size(); }
}

GenericStack<Integer> intStack = new GenericStack<>();
intStack.push(1); intStack.push(2); intStack.push(3);
System.out.println(intStack.pop()); // 3
```

10.2 Lambda Expressions & Functional Interfaces

Lambdas (Java 8+) let you write anonymous functions inline. Syntax: (parameters) -> expression. Used heavily with Collections sorting and Streams.

Lambda Expressions

```
// Comparator with lambda
ArrayList<String> names = new ArrayList<>(Arrays.asList("Zara","Alice","Bob","Charlie"));

// Sort ascending by length
names.sort((a, b) -> a.length() - b.length());

// Sort descending alphabetically
names.sort((a, b) -> b.compareTo(a));

// Runnable with lambda
Runnable r = () -> System.out.println("Running in thread!");
new Thread(r).start();

// Comparator.comparing – cleaner syntax
names.sort(Comparator.comparing(String::length));
names.sort(Comparator.comparing(String::length).reversed());

// Multi-key sort
// Sort students by grade desc, then name asc
students.sort(
    Comparator.comparingInt((Student s) -> s.grade).reversed()
    .thenComparing(s -> s.name)
);
```

10.3 Java Streams — Quick Overview

Streams (Java 8+) process collections in a declarative, pipeline style. `filter()` → `map()` → `collect()` is the most common pattern.

Java Streams

```
import java.util.stream.*;

List<Integer> numbers = Arrays.asList(1,2,3,4,5,6,7,8,9,10);

// Sum of even numbers
int sumEvens = numbers.stream()
    .filter(n -> n % 2 == 0)
    .mapToInt(Integer::intValue)
    .sum();

System.out.println(sumEvens); // 30

// Collect filtered list
List<Integer> evens = numbers.stream()
    .filter(n -> n % 2 == 0)
    .collect(Collectors.toList());

// Map: double each element
List<Integer> doubled = numbers.stream()
```



```
.map(n -> n * 2)
.collect(Collectors.toList());

// Count elements matching condition
long count = numbers.stream().filter(n -> n > 5).count();

// Find max
Optional<Integer> max = numbers.stream().max(Integer::compareTo);
System.out.println(max.get()); // 10

// Sort and collect
List<String> sorted = names.stream()
    .filter(s -> s.length() > 3)
    .sorted()
    .collect(Collectors.toList());

// String joining
String joined = names.stream().collect(Collectors.joining(", "));
```

CH 11

More Practice Problems

15+ additional problems with solutions — all HackerRank style

Q14. Pascal's Triangle

```
public static void printPascal(int rows) {
    int[][] triangle = new int[rows][];
    for (int i = 0; i < rows; i++) {
        triangle[i] = new int[i + 1];
        triangle[i][0] = 1;
        triangle[i][i] = 1;
        for (int j = 1; j < i; j++)
            triangle[i][j] = triangle[i-1][j-1] + triangle[i-1][j];
    }
    for (int[] row : triangle) {
        for (int val : row) System.out.print(val + " ");
        System.out.println();
    }
}

// Row 0: 1
// Row 1: 1 1
// Row 2: 1 2 1
// Row 3: 1 3 3 1
```

Q15. GCD and LCM

```
// Euclidean algorithm for GCD
public static int gcd(int a, int b) {
    while (b != 0) { int t = b; b = a % b; a = t; }
    return a;
}

// LCM using GCD
public static long lcm(int a, int b) {
    return (long) a / gcd(a, b) * b; // cast first to avoid overflow
}

System.out.println(gcd(48, 18)); // 6
System.out.println(lcm(4, 6)); // 12
```

Q16. Power of a Number (Fast Exponentiation)

```
// Naive: O(n) – multiply n times
// Fast Power: O(log n) – binary exponentiation
public static long fastPow(long base, int exp) {
    long result = 1;
    while (exp > 0) {
        if ((exp & 1) == 1) result *= base; // if exp is odd
        base *= base;
        exp >>= 1; // exp = exp / 2
    }
    return result;
}

// fastPow(2, 10) -> 1024 | fastPow(3, 5) -> 243
```

Q17. Sieve of Eratosthenes — All Primes up to N

```
public static boolean[] sieve(int n) {
    boolean[] isPrime = new boolean[n + 1];
    Arrays.fill(isPrime, true);
    isPrime[0] = isPrime[1] = false;
    for (int i = 2; i * i <= n; i++) {
        if (isPrime[i]) {
            for (int j = i * i; j <= n; j += i)
                isPrime[j] = false;
        }
    }
    return isPrime;
}

// Print all primes up to 30
boolean[] primes = sieve(30);
for (int i = 2; i <= 30; i++)
    if (primes[i]) System.out.print(i + " ");
// 2 3 5 7 11 13 17 19 23 29
```

Q18. Merge Two Sorted Arrays Without Extra Space

```
// Merge two sorted arrays into one sorted array
public static int[] mergeSorted(int[] a, int[] b) {
    int i = 0, j = 0, k = 0;
    int[] result = new int[a.length + b.length];
    while (i < a.length && j < b.length) {
        if (a[i] <= b[j]) result[k++] = a[i++];
        else result[k++] = b[j++];
    }
    while (i < a.length) result[k++] = a[i++];
    while (j < b.length) result[k++] = b[j++];
}
```

```
return result;  
}  
// mergeSorted([1,3,5], [2,4,6]) -> [1,2,3,4,5,6]
```

Q19. Remove Duplicates from Sorted Array (In-Place)

```
public static int removeDuplicates(int[] arr) {
    if (arr.length == 0) return 0;
    int k = 1; // pointer to place next unique element
    for (int i = 1; i < arr.length; i++) {
        if (arr[i] != arr[i - 1]) {
            arr[k++] = arr[i];
        }
    }
    return k; // new length
}

// [1,1,2,3,3,4] -> arr becomes [1,2,3,4,...] returns 4
```

Q20. Validate an Email Address

```
public static boolean isValidEmail(String email) {
    String regex = "^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\\.[a-zA-Z]{2,}$";
    return email != null && email.matches(regex);
}

System.out.println(isValidEmail("nitesh@gmail.com")); // true
System.out.println(isValidEmail("nitesh@.com")); // false
System.out.println(isValidEmail("nitesh@gmail")); // false
```

Q21. Find Missing Number in 1 to N

```
// Using sum formula: sum of 1..n = n*(n+1)/2
public static int missingNumber(int[] arr, int n) {
    long expected = (long) n * (n + 1) / 2;
    long actual = 0;
    for (int x : arr) actual += x;
    return (int)(expected - actual);
}

// arr=[1,2,4,5,6], n=6 -> missing = 21-18 = 3

// XOR approach - works even with large numbers
public static int missingXOR(int[] arr, int n) {
    int xor = 0;
    for (int i = 1; i <= n; i++) xor ^= i;
    for (int x : arr) xor ^= x;
    return xor;
}
```

Q22. Implement a Simple Calculator Using OOP

```
public class Calculator {
    private double memory = 0;

    public double calculate(double a, char op, double b) {
```

```
switch (op) {
case '+': return a + b;
case '-': return a - b;
case '*': return a * b;
case '/': {
if (b == 0) throw new ArithmeticException("Division by zero");
return a / b;
}
case '%': return a % b;
default: throw new IllegalArgumentException("Unknown op: " + op);
}

public void storeMemory(double val) { this.memory = val; }
public double recallMemory() { return this.memory; }

public static void main(String[] args) {
Calculator calc = new Calculator();
System.out.println(calc.calculate(10, '+', 5)); // 15.0
System.out.println(calc.calculate(10, '/', 3)); // 3.333...
calc.storeMemory(42);
System.out.println(calc.recallMemory()); // 42.0
}
}
```

Q23. Abstract Shape Area — Polymorphism Problem

This is the most common OOP HackerRank template. Know it cold.

```
import java.util.Scanner;

abstract class Shape {
    abstract double area();
    abstract double perimeter();
    public String toString() {
        return String.format("Area=%.2f Perimeter=%.2f", area(), perimeter());
    }
}

class Circle extends Shape {
    double r;
    Circle(double r) { this.r = r; }
    double area() { return Math.PI * r * r; }
    double perimeter() { return 2 * Math.PI * r; }
}

class Rectangle extends Shape {
    double l, w;
    Rectangle(double l, double w) { this.l = l; this.w = w; }
    double area() { return l * w; }
    double perimeter() { return 2 * (l + w); }
}

class Triangle extends Shape {
    double a, b, c;
    Triangle(double a, double b, double c) { this.a=a; this.b=b; this.c=c; }
    double area() {
        double s = (a + b + c) / 2;
        return Math.sqrt(s*(s-a)*(s-b)*(s-c)); // Heron's formula
    }
    double perimeter() { return a + b + c; }
}

public class Main {
    public static void main(String[] args) {
        Shape[] shapes = {
            new Circle(5),
            new Rectangle(4, 6),
            new Triangle(3, 4, 5)
        };
        for (Shape s : shapes)
            System.out.println(s.getClass().getSimpleName() + ": " + s);
    }
}
```

```
}  
  
// Circle: Area=78.54 Perimeter=31.42  
// Rectangle: Area=24.00 Perimeter=20.00  
// Triangle: Area=6.00 Perimeter=12.00
```

Q24. Interface — Multiple Behaviour Implementation

```
interface Flyable { void fly(); }  
interface Swimmable { void swim(); }  
interface Walkable { void walk(); }  
  
class Duck implements Flyable, Swimmable, Walkable {  
    String name;  
    Duck(String name) { this.name = name; }  
    public void fly() { System.out.println(name + " is flying!"); }  
    public void swim() { System.out.println(name + " is swimming!"); }  
    public void walk() { System.out.println(name + " is walking!"); }  
}  
  
Duck d = new Duck("Donald");  
Flyable f = d; f.fly(); // Donald is flying!  
Swimmable s = d; s.swim(); // Donald is swimming!  
// Duck can be referenced as ANY of its interfaces – polymorphism
```


CH 12

Mock HackerRank Test

3 problems · 60 minutes · Simulate the real test environment

WARNING: Close everything. Set a 60-minute timer. Attempt all 3 problems. Save after each. Do not look at answers until the timer ends. This is your most important prep step.

MOCK TEST — Problem 1 (Easy — 15 min)

Student Grade System

Create a Student class with name (String), marks (int[]). Implement: (a) average() — returns average of marks as double. (b) grade() — returns 'A' if avg>=90, 'B' if>=80, 'C' if>=70, 'D' if>=60, else 'F'. (c) topScore() — returns the highest mark. In main(), read N students from input. For each student, print: name, average, grade, top score.

Sample Input:

```
3
Alice 85 90 78 92
Bob 55 60 58 62
Charlie 95 98 100 97
```

Sample Output:

```
Alice Avg: 86.25 Grade: B Top: 92
Bob Avg: 58.75 Grade: F Top: 62
Charlie Avg: 97.50 Grade: A Top: 100
```

Solution:

```
import java.util.Scanner;

class Student {
    String name;
    int[] marks;

    Student(String name, int[] marks) {
        this.name = name; this.marks = marks;
    }

    double average() {
        int sum = 0;
        for (int m : marks) sum += m;
        return (double) sum / marks.length;
    }
}
```

```
char grade() {
    double avg = average();
    if (avg >= 90) return 'A';
    if (avg >= 80) return 'B';
    if (avg >= 70) return 'C';
    if (avg >= 60) return 'D';
    return 'F';
}

int topScore() {
    int max = marks[0];
    for (int m : marks) if (m > max) max = m;
    return max;
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt(); sc.nextLine();
        for (int i = 0; i < n; i++) {
            String[] parts = sc.nextLine().trim().split("\\s+");
            String name = parts[0];
            int[] marks = new int[parts.length - 1];
            for (int j = 1; j < parts.length; j++)
                marks[j-1] = Integer.parseInt(parts[j]);
            Student s = new Student(name, marks);
            System.out.printf("%-10s Avg: %.2f Grade: %c Top: %d\n",
                s.name, s.average(), s.grade(), s.topScore());
        }
        sc.close();
    }
}
```

MOCK TEST — Problem 2 (Medium — 20 min)

Word Frequency Counter

Read a paragraph from input (multiple lines until an empty line). Count the frequency of each word (case-insensitive, ignore punctuation). Print the top 5 most frequent words in descending order of frequency. If frequencies tie, sort alphabetically.

Sample Input:

```
Java is great. Java is fast.  
Python is great. Java and Python are great.
```

Sample Output:

```
great: 3  
java: 3  
is: 3  
python: 2  
and: 1
```

Solution:

```
import java.util.*;  
  
public class WordFreq {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        HashMap<String, Integer> freq = new HashMap<>();  
  
        String line;  
        while (sc.hasNextLine()) {  
            line = sc.nextLine();  
            if (line.trim().isEmpty()) break;  
            // Remove punctuation, lowercase, split  
            String[] words = line.toLowerCase()  
                .replaceAll("[^a-z\\s]", "")  
                .trim().split("\\s+");  
            for (String w : words)  
                if (!w.isEmpty())  
                    freq.put(w, freq.getOrDefault(w, 0) + 1);  
        }  
  
        // Sort by frequency desc, then alphabetically asc  
        List<Map.Entry<String, Integer>> entries = new ArrayList<>(freq.entrySet());  
        entries.sort((a, b) -> {  
            if (!b.getValue().equals(a.getValue()))  
                return b.getValue() - a.getValue();  
            return a.getKey().compareTo(b.getKey());  
        });
```

```
int limit = Math.min(5, entries.size());
for (int i = 0; i < limit; i++)
    System.out.println(entries.get(i).getKey() + ": " + entries.get(i).getValue());

sc.close();
}
```

MOCK TEST — Problem 3 (Hard — 25 min)

Library Management System

Design a Library system using OOP. Implement: (a) Book class — title, author, ISBN, available (boolean). (b) Library class — add books, search by title (case-insensitive), borrow(ISBN) — marks unavailable, throws BookNotAvailableException if already borrowed, return(ISBN) — marks available, listAvailable() — prints all available books sorted by title. Read commands from input: ADD, SEARCH, BORROW, RETURN, LIST.

Library Management Solution

```
import java.util.*;

class BookNotAvailableException extends Exception {
    public BookNotAvailableException(String msg) { super(msg); }
}

class Book {
    String title, author, isbn;
    boolean available;

    Book(String title, String author, String isbn) {
        this.title = title; this.author = author;
        this.isbn = isbn; this.available = true;
    }

    public String toString() {
        return "[" + isbn + "] " + title + " by " + author +
            (available ? " (Available)" : " (Borrowed)");
    }
}

class Library {
    private HashMap<String, Book> books = new HashMap<>();

    public void addBook(Book b) {
        books.put(b.isbn, b);
        System.out.println("Added: " + b.title);
    }

    public void search(String query) {
        System.out.println("Search results for: " + query);
        boolean found = false;
        for (Book b : books.values()) {
            if (b.title.toLowerCase().contains(query.toLowerCase())) {
                System.out.println(" " + b);
                found = true;
            }
        }
        if (!found) System.out.println(" No books found.");
    }
}
```

```

}

public void borrow(String isbn) throws BookNotAvailableException {
    Book b = books.get(isbn);
    if (b == null) throw new IllegalArgumentException("ISBN not found: " + isbn);
    if (!b.available)
        throw new BookNotAvailableException("Book not available: " + b.title);
    b.available = false;
    System.out.println("Borrowed: " + b.title);
}

public void returnBook(String isbn) {
    Book b = books.get(isbn);
    if (b == null) { System.out.println("ISBN not found"); return; }
    b.available = true;
    System.out.println("Returned: " + b.title);
}

public void listAvailable() {
    System.out.println("Available Books:");
    books.values().stream()
        .filter(b -> b.available)
        .sorted(Comparator.comparing(b -> b.title))
        .forEach(b -> System.out.println(" " + b));
}
}

public class Main {
    public static void main(String[] args) {
        Library lib = new Library();
        lib.addBook(new Book("Clean Code", "Robert Martin", "ISBN001"));
        lib.addBook(new Book("Effective Java", "Joshua Bloch", "ISBN002"));
        lib.addBook(new Book("Java Concurrency", "Brian Goetz", "ISBN003"));
        try {
            lib.borrow("ISBN001");
            lib.borrow("ISBN001"); // throws exception
        } catch (BookNotAvailableException e) {
            System.out.println("Error: " + e.getMessage());
        }
        lib.listAvailable();
        lib.returnBook("ISBN001");
        lib.listAvailable();
    }
}

```

FIN

Day-Before Revision Checklist

Read this the night before your Synchro test

OOP Pillars

- ✓ Encapsulation: private fields + public getters/setters
- ✓ Inheritance: extends, super(), @Override
- ✓ Polymorphism: overloading (compile) vs overriding (runtime)
- ✓ Abstraction: abstract class (partial impl) vs interface (contract)

Collections Must-Knows

- ✓ ArrayList: add, get, set, remove, contains, sort — O(1) access
- ✓ HashMap: put, get, getOrDefault, containsKey, entrySet loop
- ✓ HashSet: add returns false if duplicate — use for dedupe
- ✓ LinkedList: addFirst, addLast, peekFirst, poll — use as Queue/Stack
- ✓ import java.util.*; — ALWAYS at the top

String Essentials

- ✓ Use equals() not == for String comparison
- ✓ String is immutable — use StringBuilder for concatenation in loops
- ✓ charAt, substring, split, trim, toLowerCase, replace, contains
- ✓ Integer.parseInt(), String.valueOf() for type conversions

Exception Handling

- ✓ try-catch-finally: finally ALWAYS runs
- ✓ Unchecked: NullPointerException, ArrayIndexOutOfBoundsException, NumberFormatException
- ✓ Checked: IOException — must declare throws or catch

✓ throw new Exception() to throw, throws in method signature to declare

Sorting & Searching

- ✓ Arrays.sort(arr) — $O(n \log n)$, use this in tests
- ✓ Collections.sort(list, (a,b) -> a-b) — ascending
- ✓ Binary search only on sorted arrays — $O(\log n)$
- ✓ Know Bubble/Selection/Insertion conceptually for MCQs

DSA Patterns

- ✓ Two-pointer: pair sum, palindrome check, merge sorted arrays
- ✓ Stack: balanced parentheses, next greater element
- ✓ Queue: FIFO problems, BFS traversal
- ✓ HashMap: frequency count — `getOrDefault(key, 0) + 1`
- ✓ Linked list: reverse (3 pointers), cycle (slow-fast), middle (slow-fast)

Reach Hinjewadi Phase 3 by 9:30 AM • Carry resume, photo, ID, pen • Read ALL problems first • Save after every problem • Start with the easiest